



Global Schema:

CarRentalAvailability(Vehicle#, Company, Model, Year, Location, City, DailyPrice, WeaklyPrice, Booking_Link)

Local Schemas:

Source Hertz: **Car_Info**(Reg#, Make, Model, Year, DailyPrice, Location, City)

Source Budget: **Car**(Car#, Company, Model, Year)

Price(Car#, DailyPrice, WeaklyPrice, Address, City)

Write the mappings between Global and Local schemas using both GAV and LAV.

GAV

Create View CarRentalAvailability () AS

Select Reg# AS Vehicle#, Make AS Company, Model, Year, Location, City, DailyPrice, DailyPrice*6.5 as WeaklyPrice, “ www.heartz.com ” AS Booking_Link

From Car_Info

UNION

Select Car# AS Vehicle#, Car.Company, Car.Model, Car.Year, Price.Address AS Location, Price.City, Price.DailyPrice, Price.WeeklyPrice, “ www.budget.com ” AS Booking_Link

From Car, Price

Where Car.Car#=Price.Car#

LAV

Create Source Hertz.Car_Info AS
Select Vehicle# as Reg#, Company AS Make, Model,
Year, DailyPrice, Location, City
From CarRentalAvailaibity

Create Source Budget.Car AS
Select Vehicle# as Car#, Company, Model, year
From CarRE...

Create Source Budget.Price AS
Select Vehicle# as Car#, DailyPrice, WeeklyPrice,
Loation AS Address, City
From CarRE...

Global Schema:

CarRentalAvaliability(Vehicle#, Company, Model, Year, Location, City, DailyPrice, WeaklyPrice, Booking_Link)

Local Schemas:

Source Hertz: **Car_Info**(Reg#, Make, Model, Year, DailyPrice, Location, City)

Source Budget: **Car**(Car#, Company, Model, Year)

Price(Car#, DailyPrice, WeaklyPrice, Address, City)

Write the mappings between Global and Local schemas using both GAV and LAV.

GAV

Create View CarRentalAvailability () AS
Select Reg# AS Vehicle#, Make AS Company, Model,
Year, Location, City, DailyPrice, DailyPrice*6.5 as
WeaklyPrice, “ www.heartz.com ” AS Booking_Link
From Car_Info

UNION

Select Car# AS Vehicle#, Car.Company, Car.Model,
Car.Year, Price.Address AS Location, Price.City,
Price.DailyPrice, Price.WeeklyPrice, “ www.budget.com ”
AS Booking_Link
From Car, Price
Where Car.Car#=Price.Car#

Materialization-Based Information Integration

Data Warehousing: Materializing Integrated Data for Business Intelligence

Material taken from open source

Learning objectives

- Understand materialization as an information-integration architecture.
- Design a warehouse pipeline from heterogeneous sources to conformed dimensions and materialized analytical views.
- Derive incremental maintenance rules for INSERT, DELETE and UPDATE events.
- Compare full refresh, incremental refresh, immediate/deferred maintenance and self-maintenance.
- Understand view adaptation when the view definition changes.
- Analyse freshness, source load, warehouse storage, latency and query-performance trade-offs.

Core tension: materialization makes queries fast by moving work to ingestion/maintenance time—but creates a synchronization problem.

Virtual vs materialized integration

Virtual / mediation

- Query is evaluated over current sources.
- Little redundant storage.
- Potentially high query latency.
- Source availability affects answers.
- Good for current operational data.

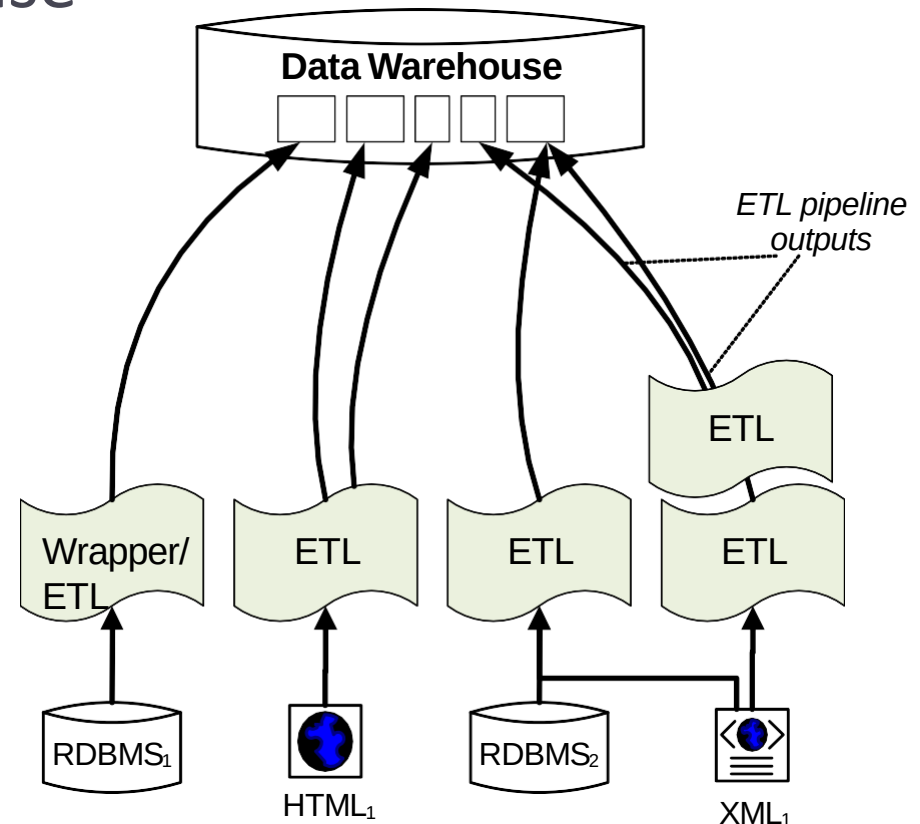
Materialized / warehouse

- Integration query is evaluated before user queries.
- Integrated result is stored.
- Fast and predictable analytical queries.
- Source outages need not block queries.
- Freshness becomes an explicit design variable.

Materialization trades runtime query cost for storage + maintenance cost.

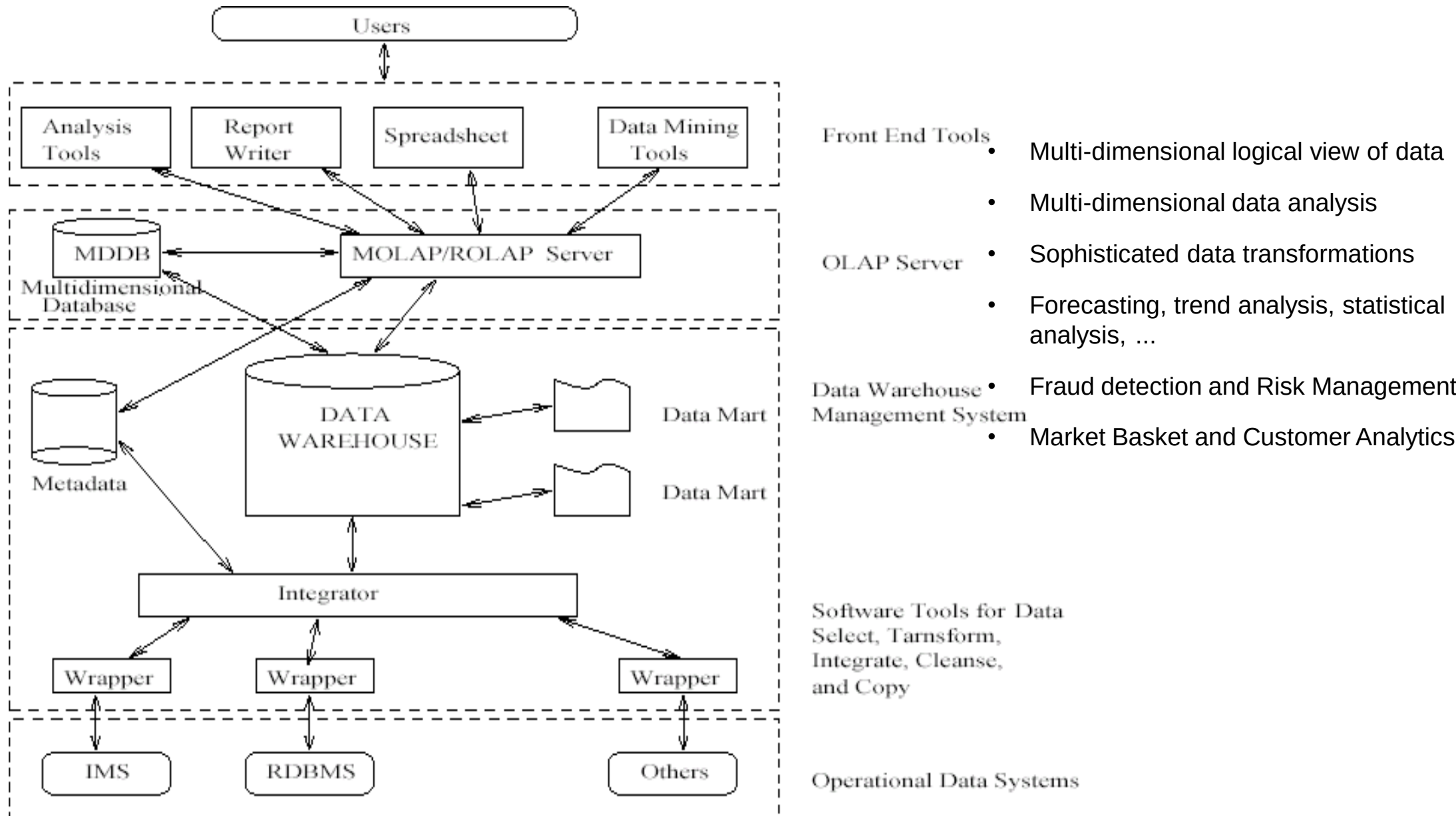
Data Warehouse (Materialization) Architecture

- At the top – a centralized database
 - Generally configured for queries and appends – not transactions
 - Many indices, materialized views, etc.
- Data is loaded and periodically updated via **Extract/Transform/Load (ETL) tools**



Data Sources → ETL → Staging → Warehouse → Data Marts

II Architecture: A Data Warehousing Approach



THINK

Where should schema matching, entity resolution and business rules happen—and which results should be materialized?

Wrapper

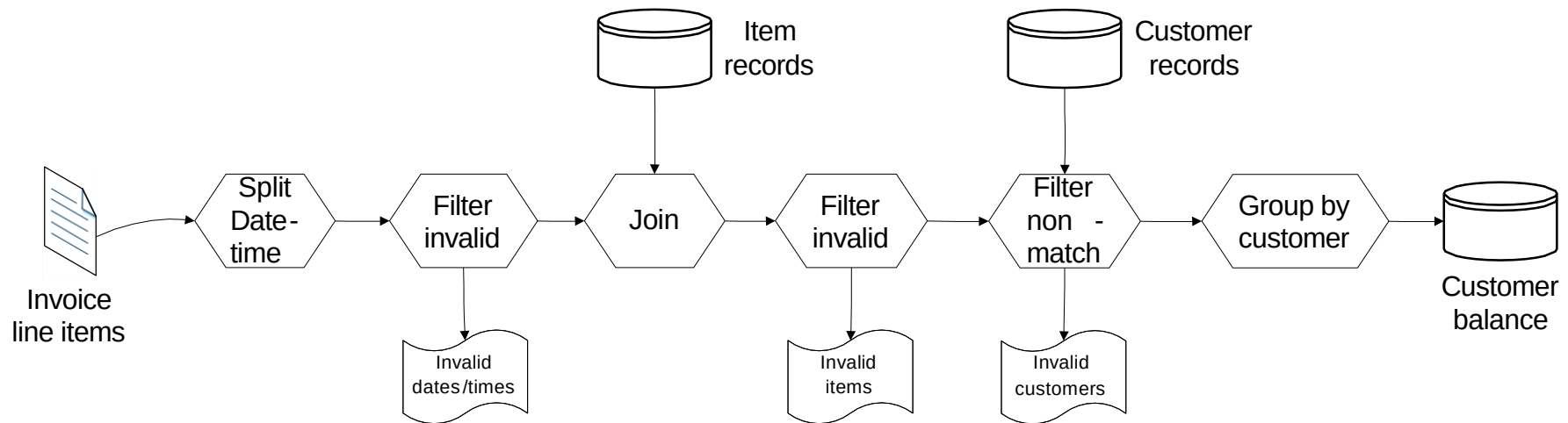


- Wrappers are components of data integration systems that communicate with the data sources
 - sending queries from higher levels in the system to the sources
 - converting replies to a format that can be manipulated by query processor
- Complexity of wrapper depends on nature of data source
 - e.g., source is RDBMS, wrapper's task is to interact with JDBC driver
 - in many cases, wrapper must parse semi-structured data such as HTML pages and transform it into a set of tuples

ETL Tools

- ETL tools are the equivalent of **schema mappings** in virtual integration, but are more powerful
- Arbitrary pieces of code to take data from a source, convert it into data for the warehouse:
 - **import filters** – read and convert from data sources
 - **data transformations** – join, aggregate, filter, convert data
 - **de-duplication** – finds multiple records referring to the same entity, merges them
 - **profiling** – builds tables, histograms, etc. to summarize data
 - **quality management** – test against master values, known business rules, constraints, etc.

Example ETL Tool Chain



- This is an example for e-commerce loading
- Note multiple stages of filtering (using selection or join-like operations), logging bad records, before we group and load

Open Source ETL Tools

- 1. Apache NiFi:** An easy-to-use data integration tool that provides a user-friendly interface for designing data flows. It supports data ingestion, transformation, and routing.
- 2. Talend Open Studio:** A comprehensive ETL tool that offers a wide range of data integration and transformation capabilities. It has a user-friendly drag-and-drop interface.
- 3. Apache Airflow:** While primarily used for orchestrating workflows, Airflow can also be used for ETL tasks through its flexible and extensible architecture.
- 4. Pentaho Data Integration (Kettle):** Part of the larger Pentaho suite, this tool allows you to visually design ETL processes and transformations.
- 5. CloverETL:** A versatile ETL tool with a visual designer and a scripting language. It can handle complex data integration and transformation tasks.
- 6. Scriptella:** A lightweight ETL framework that uses XML or YAML files to define data extraction and transformation tasks.
- 7. Jaspersoft ETL (Talend Data Integration):** Originally known as Talend Open Studio for Data Integration, this tool provides a comprehensive suite of ETL and data integration features.
- 8. GeoKettle:** A spatial ETL tool designed for geospatial data integration and transformation tasks.
- 9. Kiba ETL:** A Ruby-based ETL framework that focuses on simplicity and extensibility.
- 10. Liquibase:** While primarily a database schema versioning tool, Liquibase can also be used for ETL-like tasks when you need to manage database changes.
- 11. Beekeeper Studio:** Though more of a database client, it also provides basic ETL features for moving data between databases.
- 12. DataJoint:** A Python-based framework for managing and manipulating scientific data, often used in research contexts.

These tools vary in terms of complexity, features, and target use cases. When choosing an open-source ETL tool, consider factors like your specific requirements, data volume, complexity of transformations, and the level of technical expertise available.

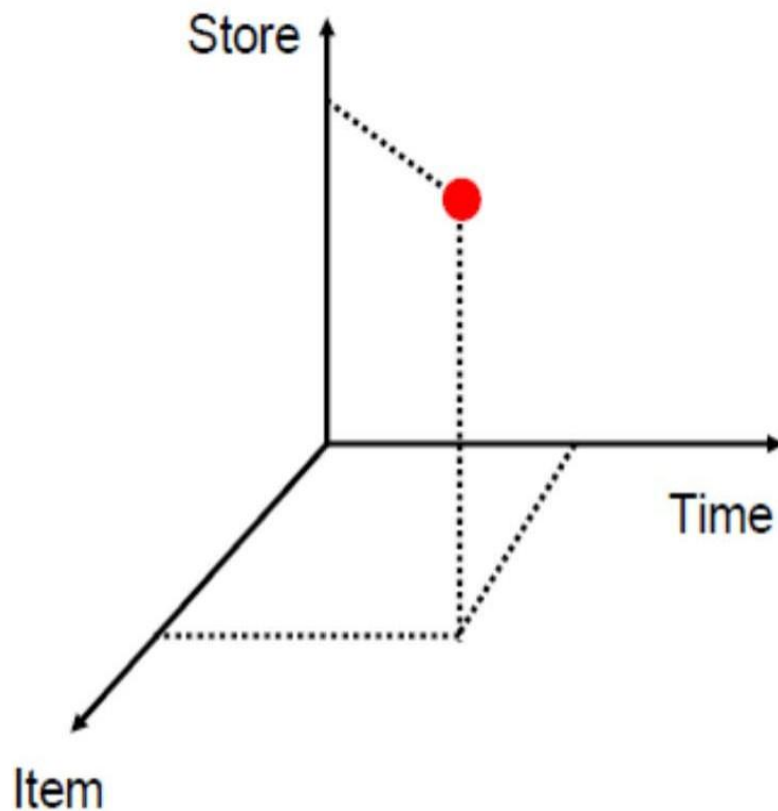
Data warehouse modeling



- Multi-dimensional data modeling
- Facts and Dimension Tables
- Entity-relationship modeling
- Temporal data modeling
- Data architecture modeling – Master Data Management

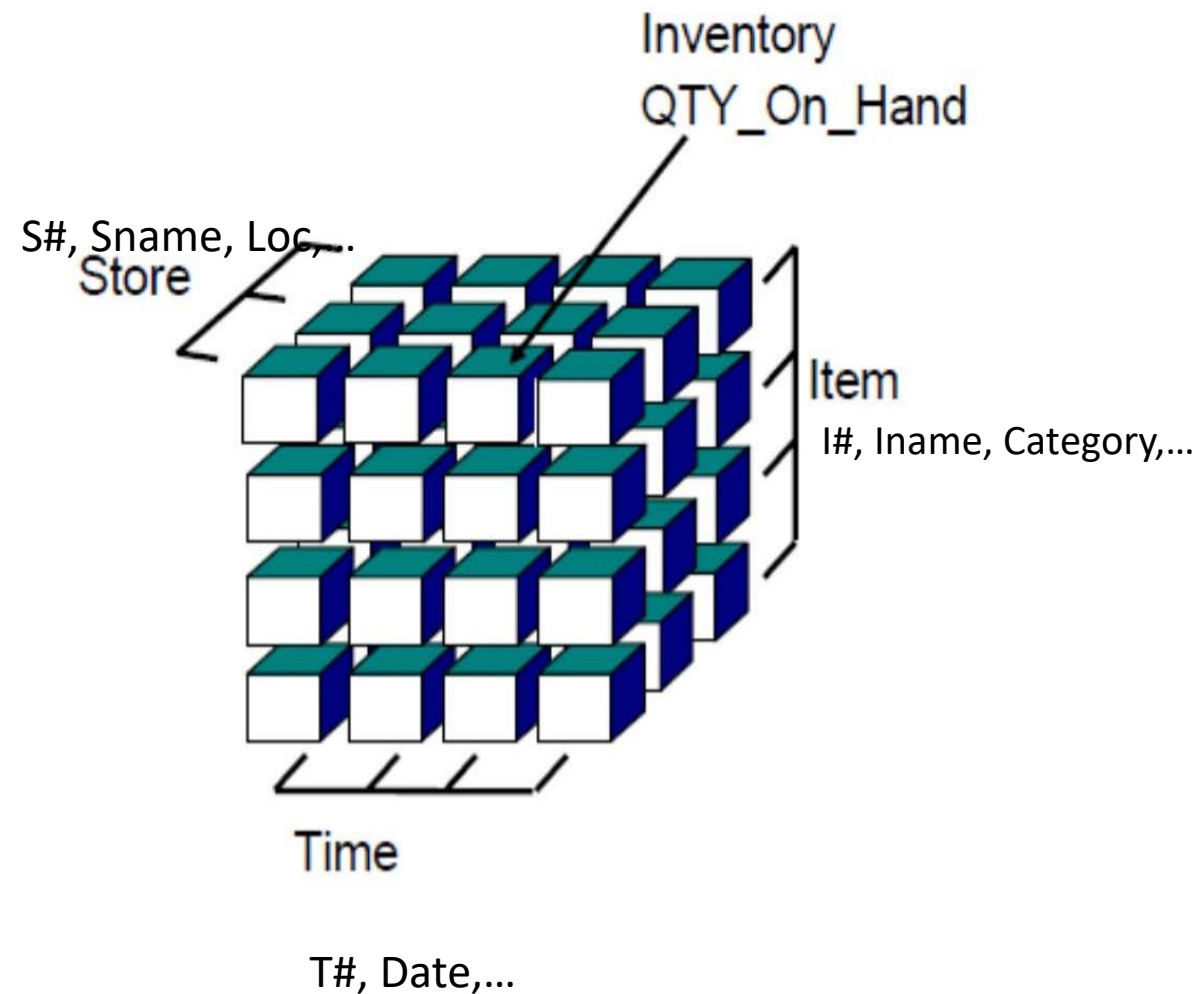
Representation of the query as a cube

(Three dimensions)



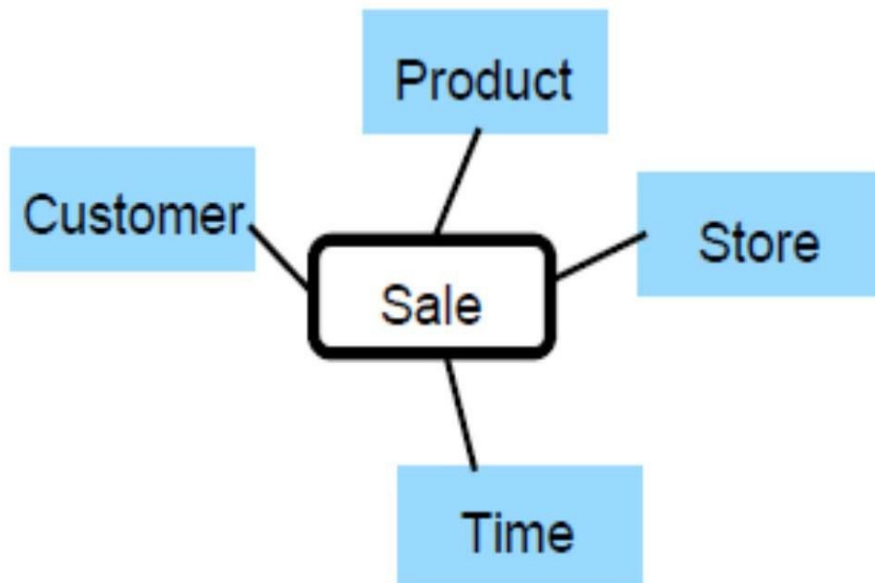
S#, I#, T#, QTY, SALES

S1 I1 T1 25 2000



Multi-dimensional data analysis

- Analysis of data along several dimensions
- Example:
 - "Analysis of sales revenue by product category, store, customer group, over the last 4 quarters"



```
select Product_Name, Store, Customer_Group, sum(Sale)  
from Sales  
group by cube(Product-Name, Store,  
               Customer_Group);
```

This computes the union of eight different groupings of the *sales* relation:

```
{ (Product_Name, Store, Customer_Group), (Product_Name,  
  Store), (Product_Name, Customer_Group), (Store,  
  Customer_Group), (Product_Name), (Store),  
  (Customer_Group), ( ) }
```

where () denotes an empty **group by** list.

- For each grouping, the result contains the null value for attributes not present in the grouping.

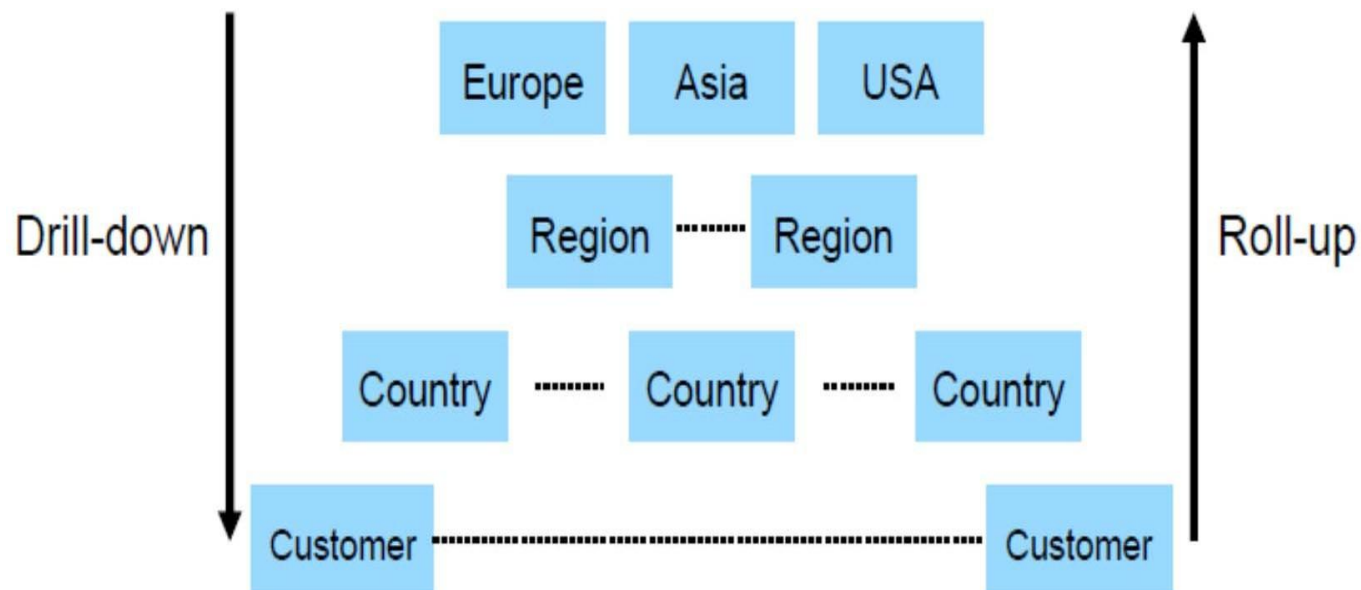
Drill-down and Roll-up

Drill-down

- Exploring facts at more detailed levels

Roll-up

- Aggregating facts at less detailed levels

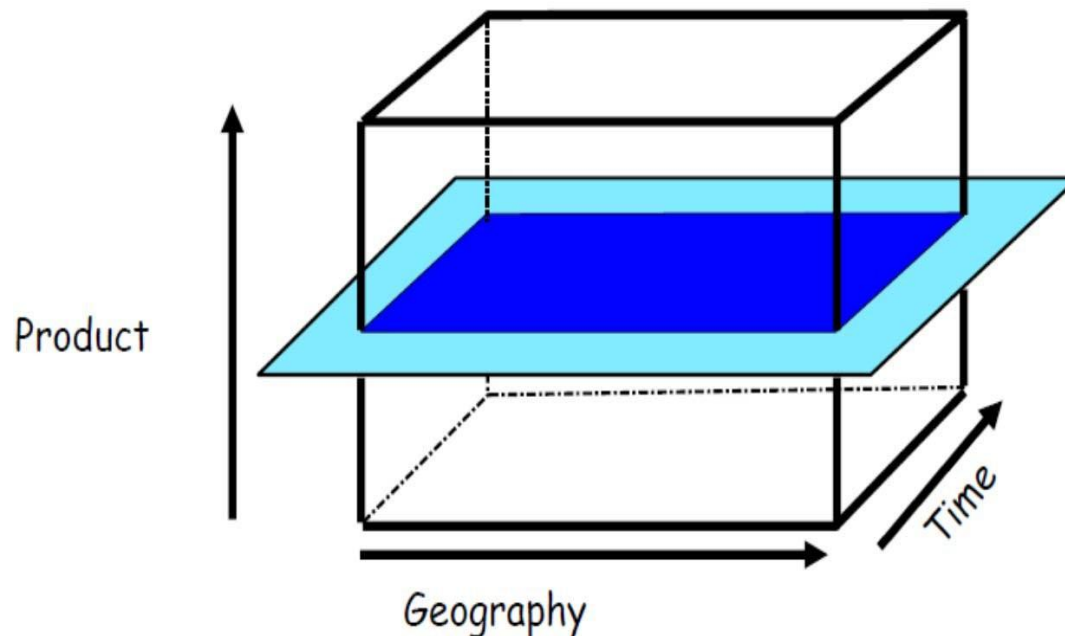


Slice, Dice, Pivot, and Drill Across analysis

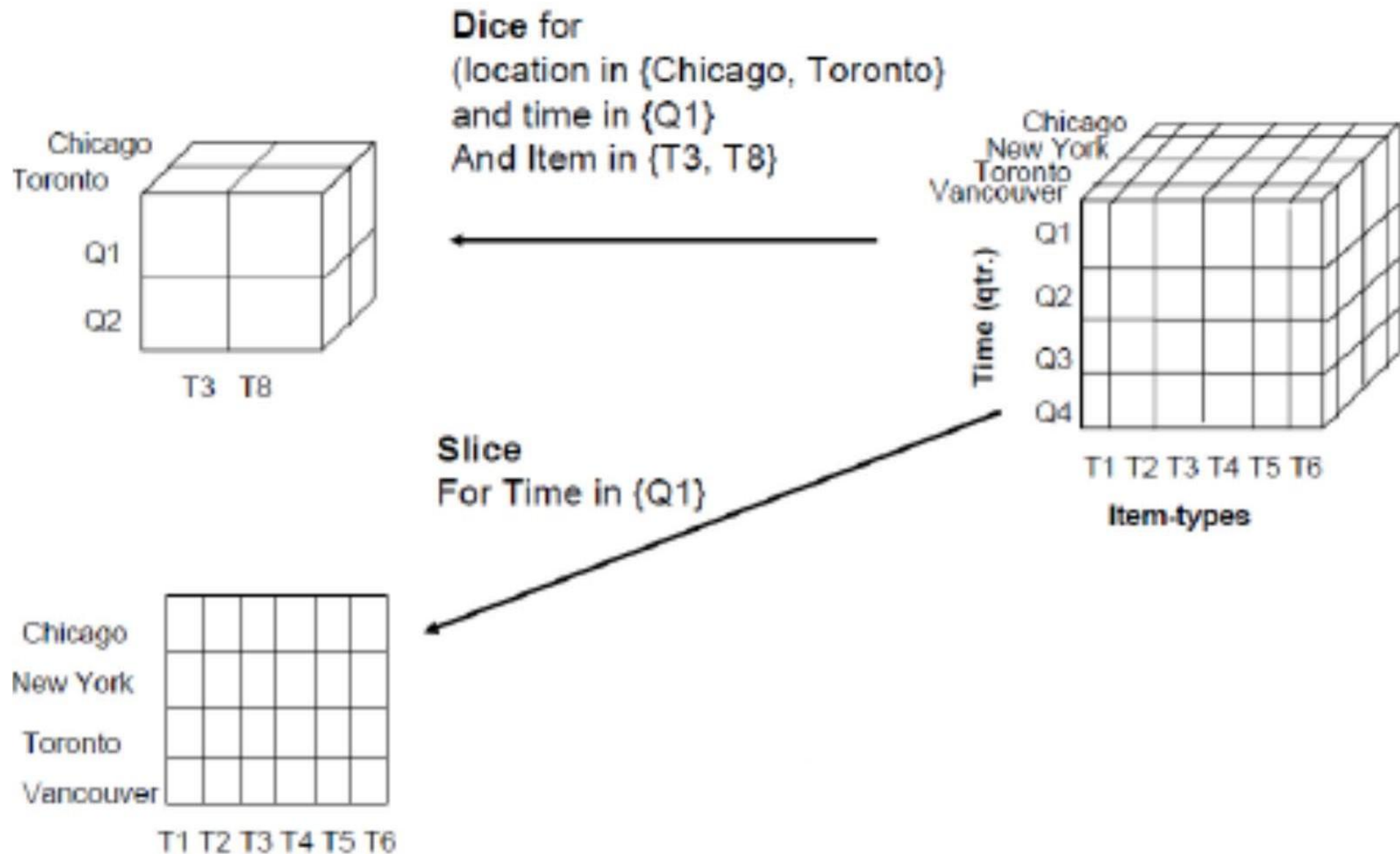
Applications

Data analysis along several dimensions and across many categories of data items, purpose is to uncover typical business behavior and rules

- Exceptional events
- Unusual activities



Slice and Dice Operation



P2P evaluation Quiz

Handover your answer sheet to your fellow classmate sitting just behind you for checking. The evaluator will write his/her Name and Std#.

When you evaluate the query, the marks should be given **if and only if** the query fetches data from *more than one data sources* and doing *aggregation/summarization on raw/integrated data*. NO PARTIAL MARKING

For each correct query (based on your judgement), give 0.5 marks. (I shall check 10-20 answer sheets randomly. If wrongly marked, the evaluator will be penalized by 2 marks.). If you have any doubt in marking, discuss with the fellow student.

Quiz – two marks

What is the output of each query over the following DB? Write the answer in tabular form.

Q1: Show annual sales by state, state-wise subtotals, and the overall sales total.

```
SELECT State, Year, SUM(Sales) AS Total-Sales FROM Sales  
GROUP BY ROLLUP(State, Year);
```

Q2: Calculate sales totals for every combination of Year and Product Category, including year-wise totals, category-wise totals, and the grand total.

```
SELECT Year, Prod-Cat, SUM(Sales) AS Total-Sales FROM Sales  
GROUP BY CUBE(Year, Prod-Cat);
```

State	Year	Prod-Cat	Sales
UP	2024	C1	20000
UP	2025	C1	40000
UP	2025	C2	10000
Delhi	2024	C1	12000
Delhi	2024	C2	20000
Bihar	2024	C1	20000
UP	2024	C2	15000



A1.

State	Year	Total-Sales
UP	2024	35000
UP	2025	50000
Delhi	2024	32000
Bihar	2024	20000
UP	Null	85000
Delhi	Null	32000
Bihar	Null	20000
Null	Null	137000

A2.

Year	Prod-Cat	Total-Sales
2024	C1	52000
2024	C2	35000
2025	C1	40000
2025	C2	10000
2024	Null	87000
2025	Null	50000
Null	C1	92000
Null	C2	45000
Null	Null	137000



View Selection and Maintenance Problems

View Selection Problem



- **DWH performance bottleneck:** frequent OLAP queries on huge fact tables.
- **Need:** Precompute & store *materialized views* (MVs).
- **Challenge:** Limited storage, update cost vs query speed.
- **Example query:** `SELECT region, SUM(sales) FROM Fact GROUP BY region;`

Possible Techniques for Choosing What to Materialize



Cache results of prior queries

- Take the results of each query, materialize them
- Use answering queries using views to reuse
- Expire using time-to-live... May not always be fresh!

Administrator-selected views

- Someone manually specifies views to compute and maintain, as with a relational DBMS
- System automatically maintains

Automatic view selection

- Using query workload, update frequencies – a view materialization wizard chooses what to materialize

Materialized View Selection – Problem Setting

- **Goal:** Choose optimal set of views to minimize query cost & storage
- **Workload-driven:** Queries define candidate aggregates
- **Constraints:** Storage budget, maintenance cost
- **NP-hard problem** (set cover / knapsack analogy)

Problem Formulation

- Input:
Workload of queries $W = \{q_1, q_2, \dots, q_n\}$
Candidate view set $V = \{v_1, v_2, \dots, v_m\}$
Storage budget S
- Cost model:
QueryCost(q, V') = cost of answering query q using views V'
Benefit(v) = $\sum$ reduction in query cost if view v is selected
- Optimization:
$$\min_{V' \subseteq V} \sum_{q \in W} \text{QueryCost}(q, V') - C_{\text{maint}}(V') \quad \text{s.t.} \quad \sum_{v \in V'} \text{Size}(v) \leq S$$

View Selection Algorithms



- Greedy Heuristic: Iteratively add view with max (benefit / cost)
- Dynamic Programming: Explore subsets (expensive for large schemas)
- Lattice Framework: Candidate space = organizes all GROUP-BY combinations
 - Star schema → lattice of aggregates
- Example: choosing between Region-level vs Region+Product aggregates.

Greedy Heuristic for View Selection

Algorithm (Greedy Benefit/Cost):

Input: Queries W , Candidate Views V , Storage S

Initialize $\text{SelectedViews} = \{\}$

While space available do

For each v in $V - \text{SelectedViews}$:

 Compute $\text{Benefit}(v) = \Sigma$ cost reduction for queries in W

 Compute $\text{Efficiency}(v) = \text{Benefit}(v) / \text{Size}(v)$

Choose $v^* = \text{argmax } \text{Efficiency}(v)$

If $\text{Size}(\text{SelectedViews} \cup \{v^*\}) \leq S$:

 Add v^* to SelectedViews

Return SelectedViews

- **Time complexity:** $O(|V| * |W| * \text{iterations})$
- Good practical performance but no global optimality guarantee

Which materialized views need to be selected in this case study, if we apply Greedy View Selection Algorithm?

Workload Queries (W)

Q1: SELECT region, SUM(sales) FROM Transactions GROUP BY region;

Q2: SELECT customer_id, SUM(sales) FROM Transactions GROUP BY customer_id;

Q3: SELECT product_id, AVG(sales) FROM Transactions GROUP BY product_id;

Candidate Views

v1: MV_region_sales (20 MB)

v2: MV_customer_sales (70 MB)

v3: MV_product_sales (50 MB)

Storage Budget & Benefits

Storage Budget (S) = 100 MB

Benefit(v1) = 200 sec/day

Benefit(v2) = 400 sec/day

Benefit(v3) = 250 sec/day

Efficiency Calculation

Efficiency(v1) = $200/20 = 10$

Efficiency(v2) = $400/70 \approx 5.7$

Efficiency(v3) = $250/50 = 5$

Selection Process

- Step 1: Pick v1 (highest efficiency = 10)
- Storage used = 20 MB
- Step 2: Pick v2 (efficiency 5.7 > v3's 5)
- Storage used = 90 MB (within 100 MB)
- Step 3: v3 cannot fit (requires 50 MB)

Final Selected Views

- ✓ v1: MV_region_sales
- ✓ v2: MV_customer_sales
- ✗ v3: MV_product_sales (not enough space)

Total Benefit = 600 sec/day

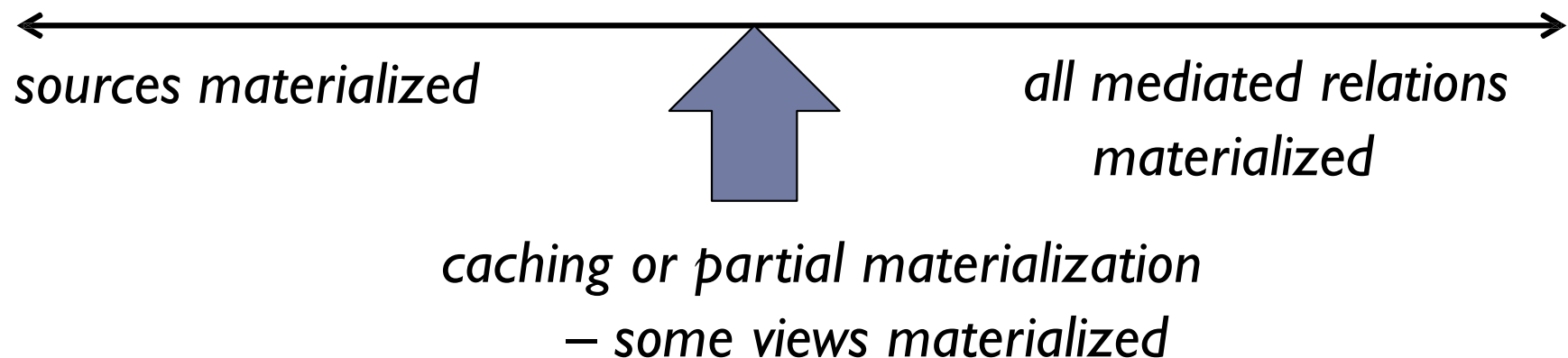
Storage Used = 90 MB

Materialized View Maintenance

The Spectrum of Materialization

Virtual integration
(EII)

Data exchange /
data warehouse



- Many real enterprise information integration systems compute and maintain materialized views, or cache results
A “hybrid” point between the fully virtual and fully materialized approaches

Think: Can we leverage Machine Learning for view recommendation?

Why materialized-view maintenance is hard

- A source tuple can affect many derived tuples through joins and GROUP BY.
- DELETES require knowing the deleted tuple's prior contribution.
- UPDATES can change join keys or grouping keys, causing delete+insert effects.
- COUNT(DISTINCT), MIN and MAX need richer state than SUM.
- Late-arriving data can invalidate already materialized aggregates.
- Distributed sources may be unavailable when maintenance needs them.
- Concurrent changes can make naïve refresh algorithms inconsistent.
- Source and warehouse transactions may have different commit times.

Research question: how can we compute ΔV without recomputing V ?

Refresh (Materialized Views update) strategies

Full refresh
Recompute V

Incremental
Compute ΔV

Deferred
Accumulate
changes

Hybrid
Incremental +
reconciliation

THINK

**When does full refresh become cheaper than incremental refresh?
Consider change volume, join selectivity, indexing and compute cost.**

Full vs Incremental view maintenance — core idea

Naïve

$$V = Q(R)$$

After ΔR :

$$V' = Q(R \cup \Delta R)$$

Recompute Q over all R .

Incremental

$$V' = V \oplus \Delta V$$

// $\oplus$ denotes the appropriate set/bag update operation //

Compute ΔV from:

$V + \Delta R$ + auxiliary state.

Goal: avoid scanning unaffected R .

Incremental Maintenance Example

View $V = \text{SELECT region, SUM(sales) FROM fact_sales GROUP BY region}$

Base update: (Delhi, Shoes, 10), Incremental delta: $\Delta V = \{(Delhi, +10)\}$

Update rule: If Delhi in V : $V[Delhi] = V[Delhi] + 10$ Else: Insert (Delhi, 10) into V

Full refresh

- Simple correctness model.
- Easy recovery from inconsistencies.
- Cost depends on full source/view size.
- Can create long refresh windows.

Incremental refresh

- Cost depends mainly on changed data.
- Requires delta rules and metadata.
- Can be dramatically faster for small changes.
- Harder for joins, deletes and non-additive aggregates.

Exercise — derive incremental maintenance rules

- Given $V = \text{Customer} \bowtie \text{Order} \bowtie \text{OrderLine}$, derive ΔV for an INSERT into OrderLine.
- Then derive ΔV for DELETE of an OrderLine.
- Now handle UPDATE of customer segment.
- Identify which existing warehouse state is required.
- State assumptions about keys and referential integrity.

Customer(customer_id, segment, ...)

Order(order_id, customer_id, ...)

OrderLine(order_id, product_id, quantity, price, ...)

Challenge: do not use a full scan of the base tables.

Solution --

1. INSERT into OrderLine

Suppose: ΔOL is the set of newly inserted OrderLine tuples.

The existing Customer and Order relations do not change.

Therefore:

$\Delta V = \text{Customer} \bowtie \text{Order} \bowtie \Delta OL$

```
In SQL: SELECT C.customer_id,  
             C.segment,  
             O.order_id,  
             OL.product_id,  
             OL.quantity,  
             OL.price  
FROM Customer C, Order O, Delta_OrderLine OL  
WHERE C.customer_id = O.customer_id, ON O.order_id = OL.order_id;
```

2. Same for DELETION

3. UPDATE: DELECTION followed by INSERTION

Think: Why can't we just update one row?

Delta rule for joins

View

$$V = R \bowtie S$$

R receives inserted tuples ΔR .

Exact insertion delta

$$\Delta V = \Delta R \bowtie S$$

If S receives ΔS :

$$\Delta V = R \bowtie \Delta S$$

If both change:

$$(R + \Delta R) \bowtie (S + \Delta S)$$

contains cross terms.

Key idea: a new tuple joins with existing tuples on the other side.

Delta rule for aggregates

SUM

$V(\text{category}) = \text{SUM}(\text{amount})$
GROUP BY category

Insert delta ΔR :

$\Delta \text{SUM}(\text{category}) = \Sigma \text{ amount in } \Delta R$

$V'[\text{category}] = V[\text{category}] + \Delta \text{SUM}(\text{category})$

For DELETE: subtract the old contribution.

SUM is easy because contributions are additive.

Why COUNT(DISTINCT) is harder

SUM

- DELETE amount $x \rightarrow$ subtract x .
- Only current group aggregate is needed.
- State can be compact.

COUNT(DISTINCT)

- Delete value x does not necessarily reduce the count.
- Need to know whether another row still has x .
- Maintain per-value multiplicity or equivalent auxiliary state.

The algebra of the aggregate determines the required maintenance state.

Making Views Self-Maintainable



- **Self-Maintainable Views:** Updated without base data
- **Auxiliary Views:** Help maintain other views efficiently

Self-maintainability

Definition

- A materialized view is self-maintainable if it can be maintained at the warehouse without accessing all base source data.
- Useful when sources are remote, expensive or unavailable.
- May require auxiliary relations stored at the warehouse.

Design trade-off

- More auxiliary data → more storage and maintenance.
- Less auxiliary data → more remote source access.
- Goal: enough information to derive exact deltas locally.
- Keys and referential constraints can reduce auxiliary data.

Self-maintenance converts network/source dependency into local storage dependency.

Auxiliary state — intuitive example

Need

$V = \text{Order} \bowtie \text{Customer}$

If `Customer.segment` changes, we need to know which orders belong to that customer.

Possible auxiliary state

Store `customer_id` → affected order IDs
or retain required join keys / attributes.

Then a segment update can identify affected `V` rows without scanning all `Orders`.

Auxiliary state is not “extra data for free”; it is a deliberate maintenance index.

Self-Maintainable vs Non-Self-Maintainable



- Self-maintainable: All updates can be computed from base relations only
 - Simple aggregate views (SUM, COUNT)
- Non-self-maintainable: Need external queries
 - Views with nested queries, NOT EXISTS clauses
- What about AVG, MAX and MIN?

Example -- Incremental Maintenance of Materialized Views

R

A	B
a	b

ΔR

→

a	b _I
---	----------------

S

B	C
b	c
b _I	c

$$V = \mathbf{PJ}_{A,C} (R \mathbf{JN} S)$$

MV

A	C
a	c

View Maintenance – ‘Counting’ Algorithm

Counting Algorithm

MV

A	C	CNT
a	c	1

- Augment a counting attribute in the view schema.
- The value of this attribute for each tuple indicates the number of alternative derivations. That is, how many times the tuple t can be derived in the view.
- For each update to the base relation, the value of count attribute for tuple t is updated if it is derived.
- Use delta relations for computing the updates.
- In our previous example, For ΔR

MV

A	C	CNT
a	c	2

View Maintenance – ‘Exact Change’ Algorithm

Exact Change Algorithm

- Maintains the view without accessing the view itself.
- Uses auxiliary relations derived from the intermediate results of the view.
- Computes exact change;
 - if a deletion of tuple t from the view is produced by the algorithm, then t is guaranteed not to exist in the new view, i.e.

$$\nabla V \cap V^{new} = \phi$$

- if an insertion of tuple t to the view is produced by the algorithm, then t does not exist in the old view, i.e.

$$\Delta V \cap V^{old} = \phi$$

'Exact Change' View Maintenance Algorithm

ΔR →	R	
	A	B
	a	b
	a ₂	b
	a	b ₁
	a ₁	b

S	
C	D
b	c
b ₁	c

$V = \mathbf{PJ}_{A,D} (R \mathbf{JN} S)$	
MV	
A	D
a	c
a ₂	c

- $I_1 = \{t \mid t \in \delta R\}$
- $I_2 = \{t \in R^{new} \mid t[\bar{A}] \in \pi_{\bar{A}}(I_1)\}$
- $J_1 = \pi_{\bar{A}\bar{D}}(I_1 \bowtie_{\bar{B}=\bar{C}} S);$
- let $J_2 = \pi_{\bar{A}\bar{D}}(I_2 \bowtie_{\bar{B}=\bar{C}} S);$
- let $EC_{\delta_V} = J_1 - J_2;$

- $I_1 = \{(a, b_1), (a_1, b)\}$
- $I_2 = \mathbf{PJ}_{A,B} (R \mathbf{SJ} \Delta R)$
 $R.A = \Delta R.A$
- $I_2 = \{(a, b)\}$
- $J_1 = \{(a, c), (a_1, c)\}$
- $J_2 = \{(a, c)\}$
- $EC = \{(a_1, c)\}$

Exact Change 'Join' Algorithm

- Let the schema of R be $R(\bar{A}, \bar{B})$ and S be $S(\bar{C}, \bar{D})$.
- Let the view V be $\pi_{\bar{A}\bar{D}}(R \bowtie_{\bar{B}=\bar{C}} S)$. Additional selections and projections are easy to deal with.
- δ : Either inserion or deletion; $\triangle$: Insertion, ∇ : Deletion.
- The R^{new} and R^{old} denote the relation R before and after the updates.
- Let δR tuples be updated.
- EC_i denotes the exact change to the node i and AR_i denotes the auxiliary relation associated with node i .

Advantage of Exact Change Algorithm



This feature is important

- to ensure the correctness of the update to views defined by aggregate functions sum and count. For instance, if a tuple is inserted into such a view which is already in the view, then the value of sum and count will be erroneously modified.

In active database applications where triggers are fired by updates to the view. If these updates are not exact change updates, then the actions caused by triggers may be incorrect.

Active Rules

An active rule is composed of three components:

Event (E): Monitor - Detect - Evaluate

Condition (C): Derive - Analyze - Evaluate

Action (A): Collaborate - Integrate - Effect

Events:

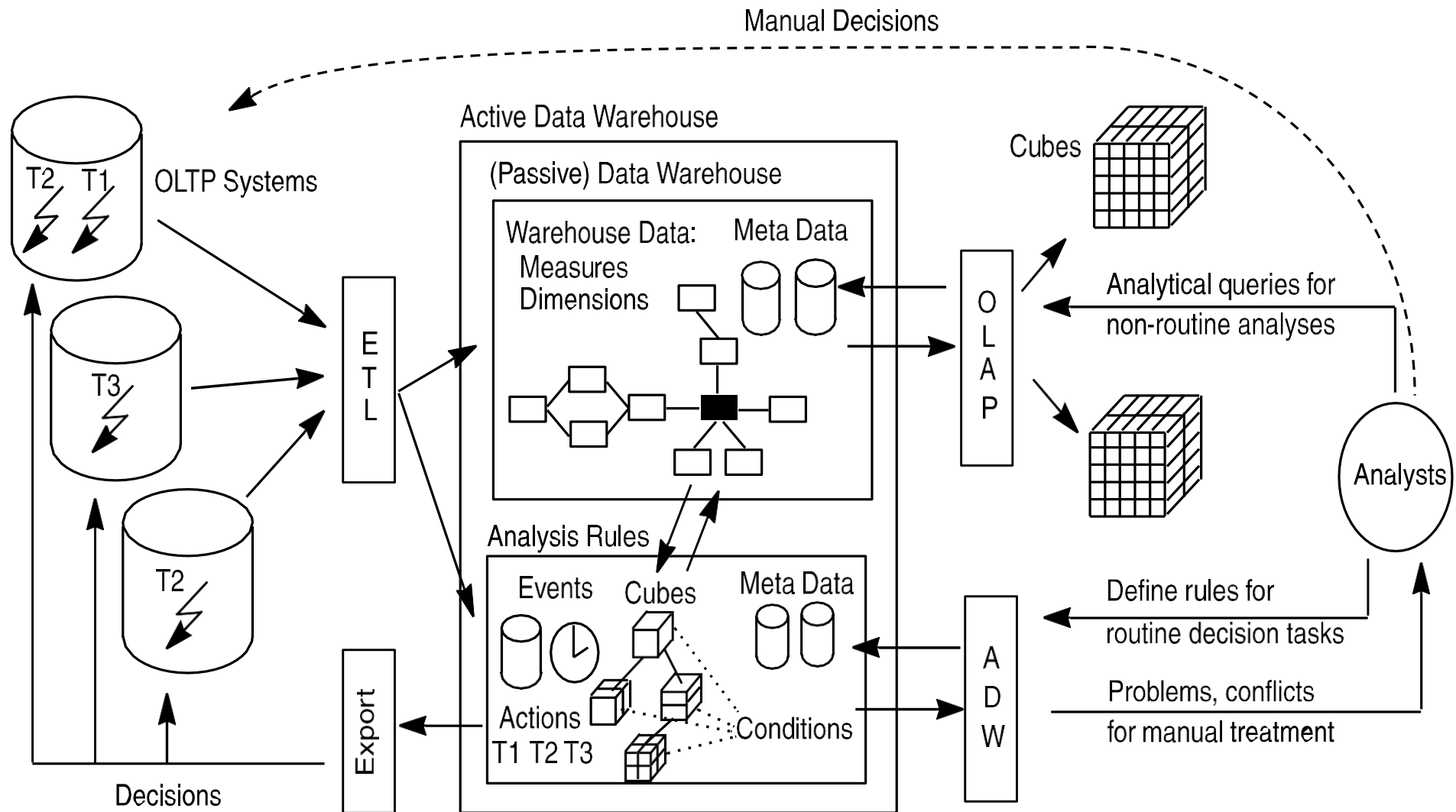
customer-event (1st purchase, new subscription, etc.)

time-based event (birthday, retirement, etc.)

product-based (launch of a new product, decline in sales, etc.)

calendar-based (Christmas, Diwali, etc.)

Active Data Warehousing System



- Real-time ETL / Change Data Capture
- Continuous Queries and Alerts
- Integration with Streaming Systems (Kafka, Spark)

View adaptation — a different problem

Maintenance

- Base data changes.
- View definition stays fixed.
- Goal: update V to remain correct.

Adaptation

- Base data may stay fixed.
- View definition changes.
- Goal: transform old materialization into the new view result.
- Can avoid full rematerialization in many cases.

This distinction is easy to miss and is an important research topic.

View adaptation example

Old view

```
V_old = SELECT category,  
            SUM(amount)  
FROM Sales  
GROUP BY category
```

New view

```
V_new = SELECT category, region,  
            SUM(amount)  
FROM Sales  
GROUP BY category, region
```

Old V lacks region → cannot
generally reconstruct V_new.

Lesson: adaptation depends on whether the old materialization retained enough information.

When adaptation is possible

Often possible

- Changing a selection predicate when required tuples remain available.
- Changing projection when omitted information is stored elsewhere.
- Some GROUP BY changes when finer-grained state was materialized.
- Union/EXCEPT changes when relevant components are retained.

Often impossible from old MV alone

- New grouping attribute was never materialized.
- New join requires source attributes not retained.
- New predicate needs information discarded by projection.
- Semantics changed beyond what old state can support.

A materialized view is also an information-loss decision.

Designing MVs as future-proof state

Naïve design

- Materialize only the exact dashboard result.
- Minimal storage today.
- Future queries may require full rematerialization.
- View redefinition becomes expensive.

Strategic design

- Materialize reusable intermediate states.
- Retain keys needed for joins.
- Retain dimensions needed for likely grouping.
- Use auxiliary views to support self-maintenance.
- Optimise the portfolio, not one view.

The best MV design balances today's query speed against tomorrow's maintenance and adaptation cost.